

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ
ĐẠI HỌC QUỐC GIA HÀ NỘI
VIỆN TRÍ TUỆ NHÂN TẠO

THỰC HÀNH PHÁT TRIỂN
TRÍ TUỆ NHÂN TẠO

CHỦ ĐỀ: HỆ THỐNG PHÂN LOẠI RÁC

Nhóm: Bombardini Gusini

Hoàng Sơn 23020423

Phạm Minh Tú 23020429

Nguyễn Công Trình 23020441

Nguyễn Công Vinh 23020445

Ngày 25, tháng 05, năm 2025

Mục lục

1	Giới thiệu	2
1.1	Mục tiêu	2
1.2	Lý do chọn đề tài	2
1.3	Phạm vi thực hiện	2
2	Công nghệ sử dụng	4
2.1	Ngôn ngữ lập trình	4
2.2	Thư viện và Framework	4
2.3	Công cụ phát triển	5
2.4	Môi trường thực thi	5
3	Chức năng và kiến trúc hệ thống	6
3.1	Các chức năng chính	6
3.2	Thiết kế ca sử dụng	7
3.2.1	Người dùng	7
3.2.2	Admin	9
3.2.3	Biểu đồ ca sử dụng	12
3.3	Biểu đồ lớp	13
3.3.1	Backend	13
3.3.2	User Frontend	17
3.3.3	Admin Frontend	21
3.4	Biểu đồ tuần tự	25
3.4.1	Người dùng	25
3.4.2	Admin	28
4	Kết luận và hướng phát triển	31
4.1	Tổng kết quá trình thực hiện	31
4.2	Kết quả đạt được	31
4.3	Các hạn chế còn tồn tại	31
4.4	Định hướng phát triển tương lai	31

Chương 1

Giới thiệu

1.1 Mục tiêu

Mục tiêu của đề tài là xây dựng một trang web hỗ trợ nhận diện rác tự động trên ảnh hoặc trên video sử dụng công nghệ học sâu (Deep Learning). Các loại rác được hỗ trợ nhận diện bao gồm:

- 0: food.
- 1: glass.
- 2: metal.
- 3: paper.
- 4: plastic.
- 5: other.

1.2 Lý do chọn đề tài

Ô nhiễm môi trường do rác thải chưa được phân loại đúng cách đang dần trở thành một vấn nạn toàn cầu, gây ra những hệ lụy nghiêm trọng đối với hệ sinh thái và đời sống con người. Nguyên nhân không chỉ xuất phát từ ý thức còn hạn chế của một bộ phận người dân, mà còn do sự thiếu hụt kiến thức và thông tin cập nhật về quy trình phân loại rác thải. Trong bối cảnh đó, Luật Bảo vệ môi trường 2020 quy định rõ: kể từ ngày 1/1/2025, mọi hộ gia đình và cá nhân bắt buộc phải thực hiện phân loại chất thải rắn sinh hoạt ngay tại nguồn.

Từ thực trạng và yêu cầu cấp bách nêu trên, nhóm đã chọn đề tài phát triển "**Hệ thống phân loại rác tự động**" ứng dụng công nghệ học sâu và triển khai trên nền tảng web.

1.3 Phạm vi thực hiện

- Chuẩn bị dữ liệu, fine-tune mô hình YOLO cho bài toán detection.

- Xây dựng giao diện web, cơ sở dữ liệu.
- Xây dựng Backend và kết nối với Frontend.
- Triển khai hệ thống trên Docker.

Chương 2

Công nghệ sử dụng

2.1 Ngôn ngữ lập trình

Frontend:

- JavaScript.
- TypeScript.
- TypeScript + JSX.
- CSS.

Backend:

- Python3.10

2.2 Thư viện và Framework

Framework chính:

- FastAPI: xây dựng API.
- SQLAlchemy: tương tác với cơ sở dữ liệu.
- Uvicorn: chạy FastAPI.
- Pytest: viết và chạy unit test.

Thư viện chính:

- ultralytics: huấn luyện và sử dụng mô hình.
- opencv-python: xử lý ảnh và video.
- pydantic: kiểm tra và xử lý dữ liệu vào/ra.
- requests: gửi HTTP request.

2.3 Công cụ phát triển

- Visual Studio Code: viết code.
- Kaggle: Huấn luyện mô hình
- Github: Quản lý mã nguồn

2.4 Môi trường thực thi

Docker: giúp chuẩn hóa môi trường và dễ dàng triển khai trên nhiều máy.

Chương 3

Chức năng và kiến trúc hệ thống

3.1 Các chức năng chính

- Người dùng tải ảnh rác lên hệ thống.
- Người dùng sử dụng webcam để tải video trực tiếp lên hệ thống.
- Hệ thống sử dụng model để phân tích, xác định các loại rác trong ảnh hoặc video.
- Kết quả trả về hiển thị trên giao diện web:
 - Upload ảnh: Kết quả trả về là ảnh đã được vẽ bounding box cùng loại rác và conf nếu nhận diện có rác trong ảnh. Nếu nhận diện trong ảnh không có rác, kết quả trả về là ảnh gốc.
 - Sử dụng Webcam: Trả về luồng video thời gian thực với các vật thể rác được nhận diện bằng bounding box, loại rác và conf. Nếu nhận diện trong frame không có rác, luồng video vẫn được hiển thị bình thường.
- Người dùng đánh giá thông qua Upvote hoặc Downvote, comment.
- Người dùng gán nhãn lại cho ảnh nếu lựa chọn Downvote.
- Người dùng xem các điểm xử lý rác gần vị trí hiện tại.
- Hệ thống lưu trữ dữ liệu đánh giá phục vụ báo cáo của admin.
- Admin đăng nhập vào hệ thống, đăng ký tài khoản admin khác.
- Admin chỉnh sửa model.
- Admin xử lý đánh giá, ảnh được gán nhãn bởi người dùng (duyet, từ chối).
- Admin quản lý dữ liệu đánh giá, ảnh được gán nhãn bởi người dùng đã được admin xử lý (tải về, xóa).
- Admin quản lý dữ liệu thống kê của model (lượt sử dụng, accuracy từ người dùng,..)

3.2 Thiết kế ca sử dụng

3.2.1 Người dùng

Nhận diện rác bằng ảnh

- **Name:** Nhận diện rác bằng ảnh
- **Actor:** User
- **Goal:** Nhận diện thành công loại rác từ ảnh mà người dùng gửi
- **Pre-condition:** Người dùng đang ở trang chính “What kind of trash is this?”
- **Main Flow:**
 1. Người dùng chọn mục tải lên file ảnh.
 2. Người dùng chọn file ảnh cần nhận diện và xác nhận.
 3. Hệ thống tiến hành nhận diện ảnh và trả về kết quả loại rác.
- **Alternative Flow:**
 3. Nếu file không đúng định dạng, hệ thống sẽ hiển thị thông báo lỗi.
- **Post-condition:** Hệ thống hiển thị kết quả loại rác được nhận diện từ ảnh mà người dùng đã gửi.

Nhận diện rác bằng webcam

- **Name:** Nhận diện rác bằng webcam
- **Actor:** User
- **Goal:** Nhận diện thành công loại rác từ webcam trực tiếp của người dùng
- **Pre-condition:**
 - Người dùng đang ở trang chính “What kind of trash is this?”
 - Người dùng đã truy cập trang “Webcam”
- **Main Flow:**
 1. Người dùng chọn sử dụng webcam.
 2. Hệ thống yêu cầu quyền truy cập vào webcam.
 3. Người dùng xác nhận cấp quyền.
 4. Hệ thống hiển thị hình ảnh trực tiếp từ webcam của người dùng.
 5. Hệ thống tiến hành nhận diện loại rác từ hình ảnh webcam.
- **Alternative Flow:**

4. Nếu người dùng từ chối quyền truy cập, hệ thống quay lại trang “Webcam”.
 5. Nếu hệ thống nhận diện được rác, hiển thị bounding box và màu sắc tương ứng với loại rác.
- **Post-condition:** Hệ thống lưu lại hình ảnh rác đã được nhận diện từ webcam của người dùng.

Đánh giá mô hình của hệ thống

- **Name:** Đánh giá mô hình của hệ thống
- **Actor:** User
- **Goal:** Đánh giá người dùng có hài lòng với hệ thống hay không và các góp ý đi kèm
- **Pre-condition:** Người dùng đang ở trang “Bình luận mô hình”
- **Main Flow:**
 1. Người dùng chọn nhập đánh giá mô hình.
 2. Người dùng chọn icon thể hiện mức độ hài lòng (hài lòng / không hài lòng).
 3. Người dùng xác nhận gửi đánh giá.
 4. Hệ thống lưu đánh giá của người dùng.
- **Alternative Flow:**
 2. Nếu người dùng nhập dòng text góp ý, hệ thống hiển thị bàn phím để người dùng nhập nội dung đánh giá.
- **Post-condition:** Hệ thống lưu lại đánh giá của người dùng và admin có thể xem đánh giá tại trang “Duyệt đánh giá”.

Đóng góp cho tập huấn luyện của mô hình bằng cách đánh nhãn cho ảnh

- **Name:** Đóng góp cho tập huấn luyện của mô hình bằng cách đánh nhãn cho ảnh
- **Actor:** User
- **Goal:** Các mẫu được người dùng đóng góp được lưu thành công
- **Pre-condition:** Người dùng đang ở trang “Đóng góp cho mô hình”
- **Main Flow:**
 1. Người dùng chọn chức năng đóng góp cho mô hình.
 2. Người dùng chọn file ảnh và xác nhận.
 3. Hệ thống hiển thị ảnh đã chọn.

4. Người dùng tiến hành đánh nhãn cho ảnh.
5. Người dùng xác nhận gửi đóng góp.
6. Hệ thống lưu lại đóng góp của người dùng.

- **Alternative Flow:**

3. Nếu file ảnh không đúng định dạng, hệ thống hiển thị thông báo lỗi.

- **Post-condition:** Hệ thống lưu trữ đóng góp của người dùng và admin có thể xem các đóng góp tại trang “Duyệt mẫu đóng góp”.

3.2.2 Admin

Thay đổi mô hình

- **Name:** Thay đổi mô hình nhận diện rác cho hệ thống
- **Actor:** Admin
- **Goal:** Mô hình được admin lựa chọn được ứng dụng thành công cho hệ thống nhận diện rác ở trang người dùng
- **Pre-condition:**
 - Admin đã đăng nhập thành công vào trang quản lý
 - Admin đang ở trong trang “Model Management”
- **Main Flow:**
 1. Admin bấm vào thanh “Select Model Version”.
 2. Hệ thống hiển thị danh sách tất cả các phiên bản model khả dụng.
 3. Admin chọn phiên bản model mong muốn.
 4. Hệ thống hiển thị các thông số cơ bản của model đã chọn.
 5. Admin bấm “Deploy Selected Model”.
 6. Hệ thống hiển thị thông báo xác nhận thay đổi mô hình.
 7. Admin xác nhận hành động.
 8. Hệ thống áp dụng model đã chọn lên hệ thống nhận diện rác tại trang người dùng.
- **Alternative Flow:**
 7. Nếu admin hủy, hệ thống đóng thông báo và giữ nguyên mô hình hiện tại.
 8. Nếu mô hình đã chọn hết hạn hoặc không hợp lệ, hệ thống hiển thị thông báo lỗi.
- **Post-condition:** Admin thay đổi mô hình thành công. Các thông số của mô hình mới sẽ được hiển thị trên trang dashboard của admin.

Duyệt các mẫu đóng góp

- **Name:** Duyệt các mẫu mà người dùng đóng góp cho tập huấn luyện của mô hình
- **Actor:** Admin
- **Goal:** Các mẫu đã được chọn sẽ được thêm vào tập huấn luyện của mô hình nhận diện của hệ thống; ngược lại, các mẫu bị từ chối sẽ bị xóa khỏi hệ thống
- **Pre-condition:**
 - Admin đã đăng nhập thành công vào trang quản lý
 - Admin đang ở trong trang “Data Moderation”
- **Main Flow:**
 1. Admin chọn “Model Version”.
 2. Hệ thống hiển thị danh sách các phiên bản mô hình khả dụng.
 3. Admin chọn phiên bản model.
 4. Hệ thống hiển thị các mẫu được người dùng đóng góp cho phiên bản model đó.
 5. Admin thực hiện thao tác tùy chọn cho từng mẫu.
 6. Hệ thống xác nhận trạng thái của mẫu.
- **Alternative Flow 1: Chọn mẫu**
 5. Admin bấm nút “Select”.
 6. Hệ thống đánh dấu mẫu là đã được chọn và thêm vào tập huấn luyện của mô hình.
- **Alternative Flow 2: Xóa mẫu**
 5. Admin bấm nút “Reject”.
 6. Hệ thống đánh dấu mẫu là đã bị từ chối.
- **Post-condition:** Các mẫu đã được duyệt (chấp nhận hoặc từ chối) sẽ bị xóa khỏi danh sách chờ xử lý. Mẫu được chọn sẽ được thêm vào tập huấn luyện của hệ thống.

Xét duyệt các bình luận đóng góp

- **Name:** Duyệt các bình luận mà người dùng đóng góp cho mô hình
- **Actor:** Admin
- **Goal:** Các bình luận đã được chọn sẽ được thêm vào cơ sở dữ liệu của mô hình; ngược lại, các bình luận bị từ chối sẽ bị xóa khỏi hệ thống
- **Pre-condition:**

- Admin đã đăng nhập thành công vào trang quản lý
- Admin đang ở trong trang “Usage & Feedback”

- **Main Flow:**

1. Admin chọn “Model Version”.
2. Hệ thống hiển thị tất cả các phiên bản mô hình khả dụng.
3. Admin chọn phiên bản model.
4. Hệ thống hiển thị các bình luận được người dùng đóng góp cho phiên bản model đó.
5. Admin bấm nút tùy chọn cho từng bình luận.
6. Hệ thống xác nhận trạng thái của bình luận.

- **Alternative Flow 1: Chọn bình luận**

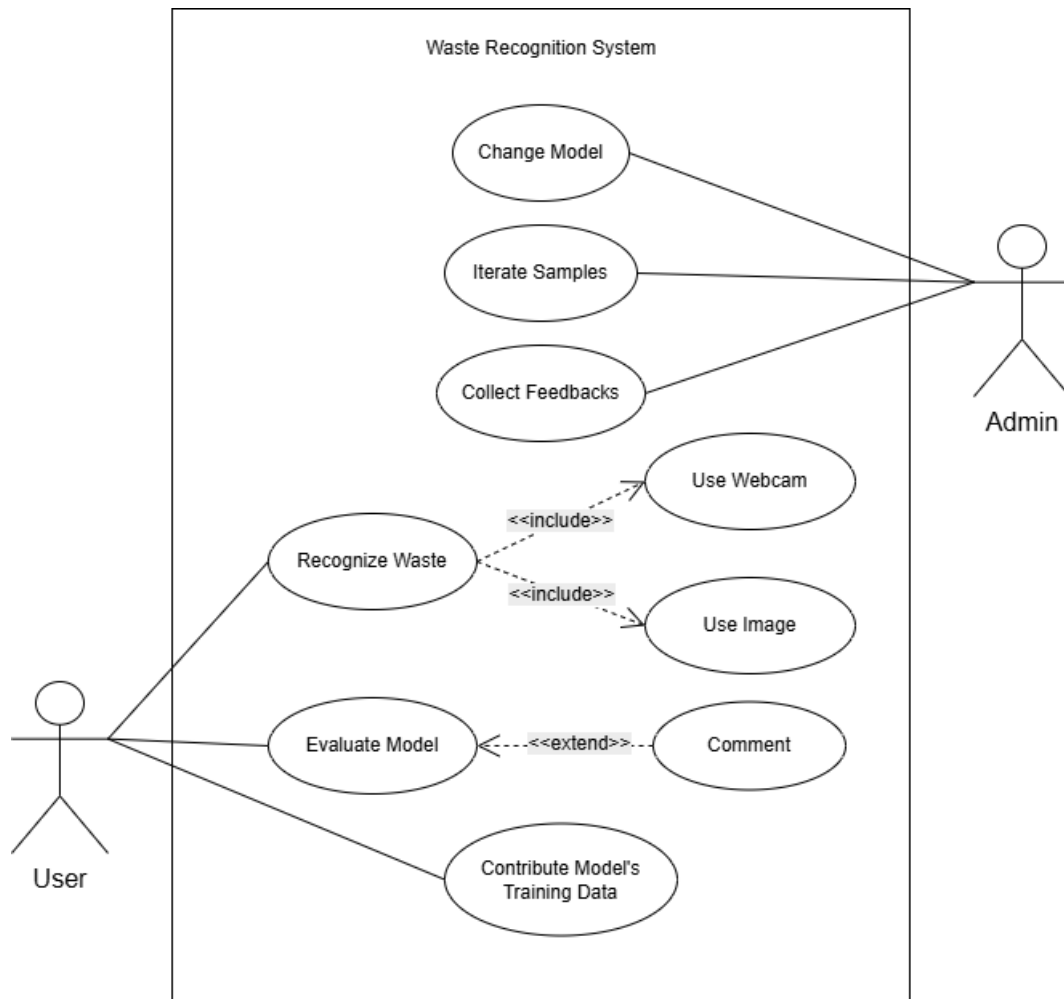
5. Admin bấm nút “Select”.
6. Hệ thống đánh dấu bình luận đã được chọn và thêm vào cơ sở dữ liệu của mô hình.

- **Alternative Flow 2: Xóa bình luận**

5. Admin bấm nút “Reject”.
6. Hệ thống đánh dấu bình luận đã bị từ chối.

- **Post-condition:** Hệ thống xóa các bình luận đã được duyệt khỏi danh sách chờ xử lý.

3.2.3 Biểu đồ ca sử dụng



Hình 3.1: Biểu đồ ca sử dụng của hệ thống

3.3 Biểu đồ lớp

3.3.1 Backend

Bảng 3.1: Danh sách các thành phần hệ thống và chức năng chính

Thành phần	Loại	Mô tả chức năng chính
Base	Class	Lớp cơ sở cho SQLAlchemy ORM.
BaseModel	Class	Lớp cơ sở kế thừa từ Pydantic cho các schema dữ liệu.
Config	Class	Lưu cấu hình hệ thống: model, đường dẫn, secret key, token expire,...
FastAPI	Class	Khởi tạo app FastAPI và cấu hình startup.
ImageRequest	Class	Schema dữ liệu ảnh đầu vào để inference.
TrashDetection	Class	Xử lý khởi tạo và nhận diện ảnh với mô hình YOLO.
WebStatistics	Class	Thống kê số lần gọi mô hình inference.
Auth	Module	Xử lý bảo mật: hash, verify password.
Database	Module	Cấu hình cơ sở dữ liệu với AsyncEngine, sessionmaker, declarative_base.
Utils	Module	Chứa các hàm tiện ích như update_env_variable, save_base64_image.
AdminAccounts	ORM Model	Thông tin tài khoản quản trị viên.
UserContributedData	ORM Model	Dữ liệu ảnh do người dùng đóng góp.
UserResponses	ORM Model	Dữ liệu phản hồi của người dùng về inference.
AdminSignUp	Pydantic	Schema đăng ký tài khoản admin.
LabelData	Pydantic	Schema nhãn: loại rác + bbox.
UserLabeledData	Pydantic	Schema dữ liệu do người dùng gán nhãn.
UserResponse	Pydantic	Schema phản hồi của người dùng.
AdminRouter	Router	Các API liên quan đến admin.
AuthRouter	Router	Đăng nhập, đăng ký, lấy thông tin admin hiện tại.
InferenceRouter	Router	Giao diện nhận ảnh để nhận diện rác.
UserRouter	Router	Nhận phản hồi hoặc nhãn từ người dùng.

Bảng 3.2: Quan hệ giữa các thành phần trong Backend

Thành phần 1	Thành phần 2	Loại quan hệ	Mô tả chi tiết
AdminRouter	AdminAccounts	Aggregation	Quản lý dữ liệu admin
AdminRouter	AsyncSession	Dependency	Cần session DB
AdminRouter	BaseModel	Dependency	Sử dụng schema kế thừa từ Pydantic
AdminRouter	Status	Dependency	Dùng enum trạng thái
AdminRouter	UserContributedData	Association	Xác minh dữ liệu người dùng
AdminRouter	UserResponses	Association	Xác minh phản hồi
Auth	JWT, password	Usage / Utility	Tạo và xác thực token
AuthRouter	AdminSignUp	Dependency	Schema đầu vào
AuthRouter	AsyncSession	Dependency	Cần session DB
AuthRouter	Auth	Usage	Dùng các hàm tiện ích về xác thực
AuthRouter	OAuth2PasswordRequestForm	Dependency	Xác thực đăng nhập
Base	AdminAccounts, UserResponses, UserContributedData	Generalization	Kế thừa lớp ORM SQLAlchemy
BaseModel	UserResponse, UserLabeledData, LabelData, AdminSignUp	Generalization	Kế thừa từ Pydantic
FastAPI	AdminRouter	Association	Gắn router admin vào app chính
FastAPI	AuthRouter	Association	Gắn router xác thực
FastAPI	Database	Dependency	Cần cấu hình DB khi khởi động app
FastAPI	InferenceRouter	Association	Gắn router inference

(tiếp trang sau)

Thành phần 1	Thành phần 2	Loại quan hệ	Mô tả chi tiết
FastAPI	UserRouter	Association	Gắn router người dùng
InferenceRouter	ImageRequest	Dependency	Dữ liệu ảnh input
InferenceRouter	TrashDetection	Dependency	Gọi mô hình YOLO
TrashDetection	Config	Dependency	Đọc thông tin cấu hình
TrashDetection	YOLO	Composition	Khởi tạo mô hình nội tại
UserLabeledData	LabelData	Composition	Dữ liệu nhãn bao gồm nhiều bbox
UserResponse	UserLabeledData	Composition	Phản hồi bao gồm nhãn ảnh
UserRouter	AsyncSession	Dependency	Cần session DB
UserRouter	UserContributed Data	Aggregation	Lưu dữ liệu ảnh người dùng gửi
UserRouter	UserLabeledData	Dependency	Schema gán nhãn người dùng
UserRouter	UserResponse	Dependency	Schema đầu vào
UserRouter	UserResponses	Aggregation	Ghi dữ liệu phản hồi người dùng
UserResponses	UserContributed Data	Association	Phản hồi xác minh dữ liệu góp phần
WebStatistics	model, count	Aggregation	Thống kê số liệu từ inference

3.3.2 User Frontend

Bảng 3.3: Danh sách các thành phần trong User Frontend

Thành phần	Loại	Mô tả chức năng chính
App	Component	Thành phần gốc bao gồm toàn bộ ứng dụng React
Header	Component	Thanh đầu trang của ứng dụng
Footer	Component	Chân trang hiển thị thông tin phụ trợ
AboutPage	Component	Trang giới thiệu nhóm phát triển
TeamMember	Data class	Lưu thông tin từng thành viên nhóm
LanguageSwitch	Component	Nút chuyển đổi ngôn ngữ
i18n	Module	Cung cấp hàm đổi ngôn ngữ (t, <code>changeLanguage</code>)
TrashDetectionPage	Component	Trang chính để nhận diện và phân loại rác
ImageUploader	Component	Cho phép người dùng tải ảnh hoặc dùng webcam
Detection	Data class	Lưu thông tin đối tượng được phát hiện trong ảnh
ColorUtils	Utility module	Chuyển đổi và ánh xạ màu theo loại rác
TrashTypeIndicator	Component	Hiển thị loại rác đã nhận diện
TrashTypeConfig	Data class	Cấu hình hiển thị từng loại rác
DisposalGuide	Data class	Hướng dẫn xử lý từng loại rác
Reference	Data class	Tài liệu tham khảo đi kèm hướng dẫn xử lý rác
ResultsDisplay	Component	Hiển thị kết quả ảnh đã xử lý
ProcessedImage	Data class	Lưu ảnh gốc, ảnh đã xử lý và kết quả nhận diện
ImageLabeler	Component	Cho phép người dùng gán nhãn lên ảnh
LabelData	Data class	Lưu thông tin nhãn (loại rác, tọa độ bbox)
FeedbackForm	Component	Gửi phản hồi về kết quả nhận diện
FeedbackData	Data class	Lưu nội dung phản hồi người dùng
RecentHistory	Component	Hiển thị các ảnh được xử lý gần đây
DisposalMap	Component	Hiển thị bản đồ gợi ý điểm vứt rác
DisposalPoint	Data class	Lưu thông tin từng điểm vứt rác (tên, vị trí, loại rác chấp nhận)

Bảng 3.4: Quan hệ giữa các thành phần trong User Frontend

Thành phần 1	Thành phần 2	Loại quan hệ	Mô tả chi tiết
App	Header, Footer, AboutPage	Composition	App là thành phần bao ngoài, bao gồm các phần giao diện chính
App	i18n	Dependency	Dùng i18n để hỗ trợ đa ngôn ngữ
LanguageSwitch	i18n	Use	Dùng hàm <code>changeLanguage()</code> để chuyển ngôn ngữ
AboutPage	TeamMember	Aggregation	Hiển thị danh sách các thành viên trong nhóm
TrashDetection Page	ImageUploader	Composition	Thành phần con để tải ảnh từ người dùng
TrashDetection Page	TrashType Indicator	Composition	Hiển thị loại rác tương ứng sau xử lý
TrashDetection Page	ResultsDisplay	Composition	Hiển thị kết quả xử lý ảnh
TrashDetection Page	ImageLabeler	Composition	Cho phép gán nhãn thủ công vào ảnh
TrashDetection Page	FeedbackForm	Composition	Thu thập ý kiến phản hồi người dùng
TrashDetection Page	RecentHistory	Composition	Hiển thị lịch sử ảnh gần đây
TrashDetection Page	ProcessedImage	Aggregation	Gắn kết ảnh đã xử lý vào page
ImageUploader	Detection	Aggregation	Kết quả nhận diện được sinh ra từ ảnh
ImageUploader	ColorUtils	Use	Sử dụng các hàm tiện ích để tô màu theo loại rác
TrashType Indicator	TrashTypeConfig	Dependency	Lấy cấu hình hiển thị của từng loại rác
TrashType Config	DisposalGuide	Aggregation	Mỗi loại rác có một hướng dẫn xử lý tương ứng
DisposalGuide	Reference	Aggregation	Hướng dẫn có thể dẫn link tham khảo

(tiếp trang sau)

Thành phần 1	Thành phần 2	Loại quan hệ	Mô tả chi tiết
ResultsDisplay	ProcessedImage	Use	Hiển thị kết quả ảnh đã xử lý
ImageLabeler	ProcessedImage	Use	Hiển thị ảnh đã xử lý để người dùng gán nhãn
ImageLabeler	LabelData	Aggregation	Mỗi nhãn gồm loại rác và bounding box
FeedbackForm	FeedbackData	Create	Tạo phản hồi từ người dùng
FeedbackData	ProcessedImage, LabelData	Association	Phản hồi liên kết với ảnh và nhãn
DisposalMap	DisposalPoint	Aggregation	Danh sách điểm vứt rác theo bản đồ
DisposalMap	ProcessedImage	Use	Dùng kết quả ảnh để xác định gợi ý vứt rác

3.3.3 Admin Frontend

Bảng 3.5: Danh sách các thành phần trong Admin Frontend

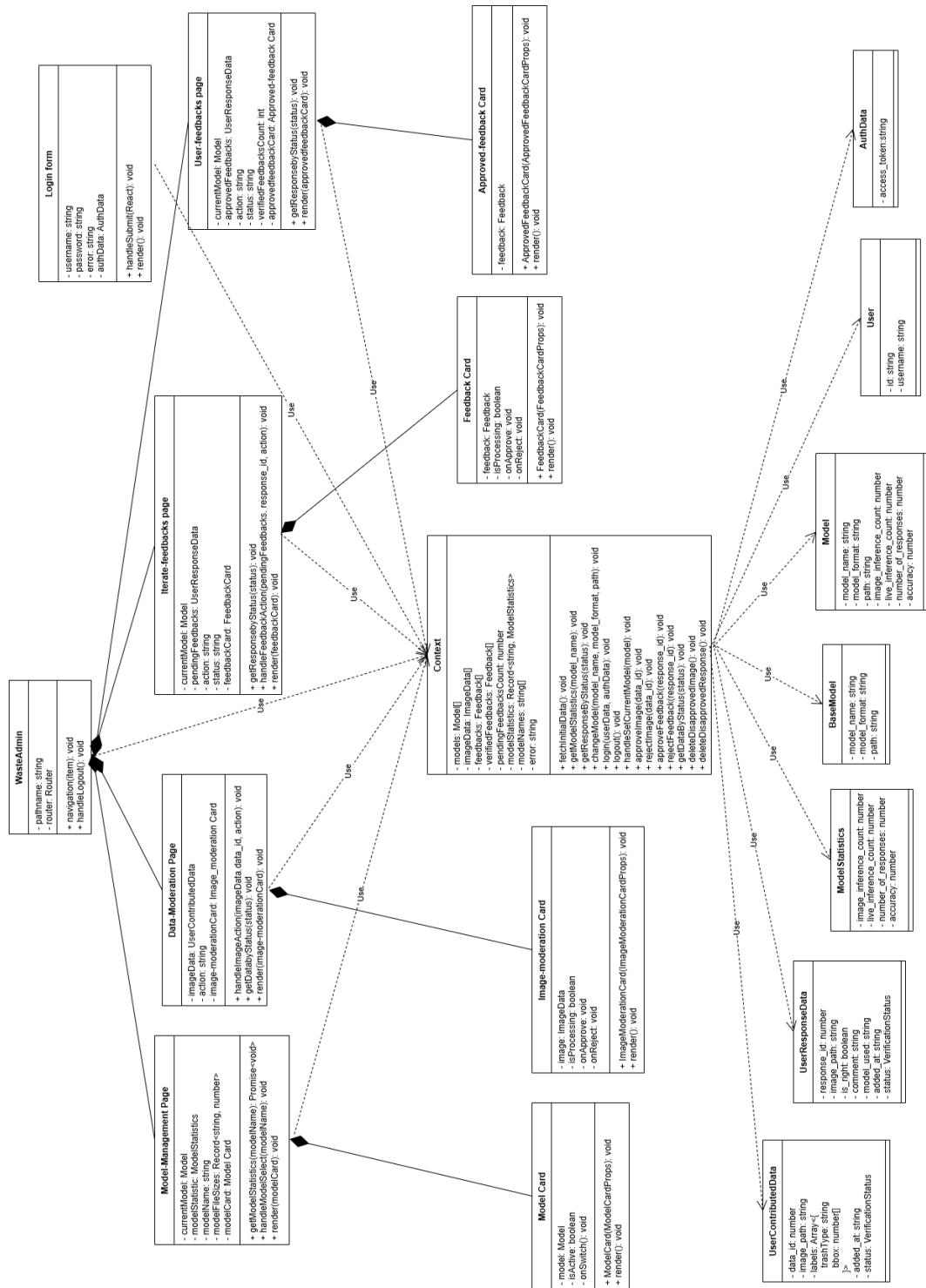
Thành phần	Loại	Mô tả chức năng chính
WasteAdmin	Component	Thành phần chính, định tuyến và điều hướng toàn hệ thống quản trị
LoginForm	Component	Giao diện đăng nhập người dùng quản trị
ModelManagementPage	Component	Trang quản lý các mô hình AI và thống kê liên quan
DataModerationPage	Component	Trang kiểm duyệt dữ liệu đóng góp từ người dùng
IterateFeedbacksPage	Component	Duyệt qua các phản hồi chưa xét duyệt
UserFeedbacksPage	Component	Hiển thị phản hồi đã duyệt
ModelCard	Component	Hiển thị thông tin từng mô hình và cho phép chuyển đổi
ImageModerationCard	Component	Hiển thị ảnh và nhãn cho kiểm duyệt thủ công
FeedbackCard	Component	Hiển thị chi tiết một phản hồi từ người dùng, cho phép phê duyệt hoặc từ chối
ApprovedFeedbackCard	Component	Hiển thị phản hồi đã được duyệt
Context	State Context	Chứa trạng thái toàn cục như danh sách mô hình, phản hồi, dữ liệu ảnh, v.v.
UserContributedData	Data class	Dữ liệu ảnh và nhãn do người dùng đóng góp
UserResponseData	Data class	Dữ liệu phản hồi người dùng đối với ảnh
Feedback	Data class	Đối tượng phản hồi người dùng, bao gồm đánh giá nhãn
ModelStatistics	Data class	Thống kê sử dụng và độ chính xác của mô hình
BaseModel	Data class	Thông tin cơ bản về mô hình
Model	Data class	Thông tin đầy đủ về mô hình AI
User	Data class	Tài khoản người dùng trong hệ thống quản trị
AuthData	Data class	Thông tin xác thực sau đăng nhập

Bảng 3.6: Quan hệ giữa các thành phần trong Admin Frontend

Thành phần 1	Thành phần 2	Loại quan hệ	Mô tả chi tiết
WasteAdmin	ModelManagement Page, DataModeration Page, IterateFeedbacks Page, UserFeedbacks Page	Composition	Điều hướng đến các trang chính của hệ thống
WasteAdmin	LoginForm	Use	Xử lý đăng nhập, đăng xuất
ModelManagement Page	ModelCard	Aggregation	Hiển thị danh sách mô hình và trạng thái
ModelManagement Page	Context	Use	Dùng để lấy và cập nhật thông tin mô hình
DataModeration Page	ImageModeration Card	Aggregation	Hiển thị ảnh cần kiểm duyệt
DataModeration Page	UserContributed Data	Use	Lấy dữ liệu ảnh từ người dùng
DataModeration Page	Context	Use	Gửi hành động kiểm duyệt lên server
IterateFeedbacks Page	FeedbackCard	Aggregation	Hiển thị phản hồi cần duyệt
IterateFeedbacks Page	UserResponseData	Use	Duyệt phản hồi chưa được xử lý
IterateFeedbacks Page	Context	Use	Lấy dữ liệu và gửi hành động
UserFeedbacks Page	ApprovedFeedback Card	Aggregation	Hiển thị phản hồi đã được duyệt
UserFeedbacks Page	UserResponse Data	Use	Lấy phản hồi đã xử lý từ người dùng
UserFeedbacks Page	Context	Use	Lấy trạng thái phản hồi từ backend
ImageModeration Card	ImageData	Use	Hiển thị ảnh từ người dùng

(tiếp trang sau)

Thành phần 1	Thành phần 2	Loại quan hệ	Mô tả chi tiết
ImageModeration Card	Context	Use	Xác nhận hoặc từ chối dữ liệu ảnh
FeedbackCard	Feedback	Aggregation	Dữ liệu phản hồi cần xử lý
FeedbackCard	Context	Use	Phê duyệt hoặc từ chối phản hồi
Approved FeedbackCard	Feedback	Use	Hiển thị phản hồi đã duyệt
Context	Model, Feedback, ImageData, ModelStatistics, UserResponse Data	Aggregation	Trạng thái trung tâm dùng toàn hệ thống
Model	BaseModel	Inheritance	Kế thừa thông tin cơ bản mô hình
LoginForm	AuthData	Use	Gửi thông tin đăng nhập và nhận token
AuthData	User	Association	Dữ liệu xác thực tương ứng với người dùng

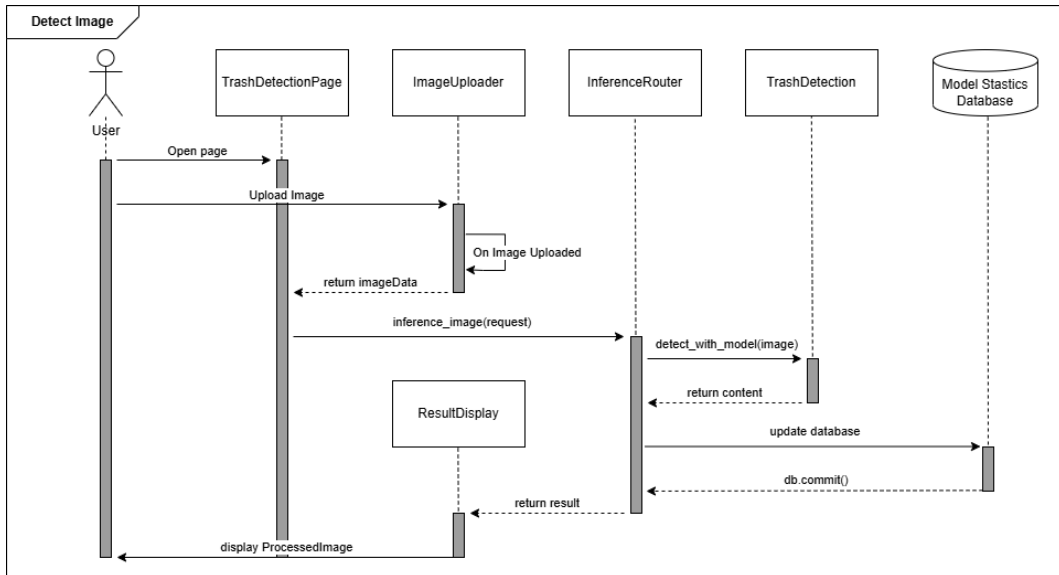


Hình 3.4: Biểu đồ lớp cho Frontend Admin

3.4 Biểu đồ tuần tự

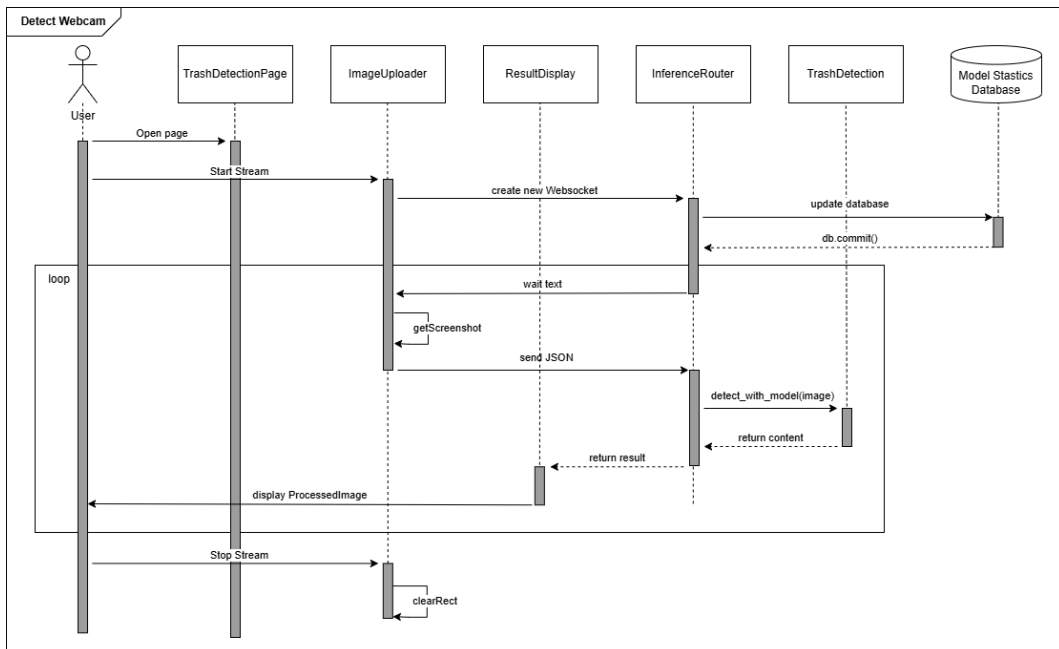
3.4.1 Người dùng

Nhận diện rác từ ảnh



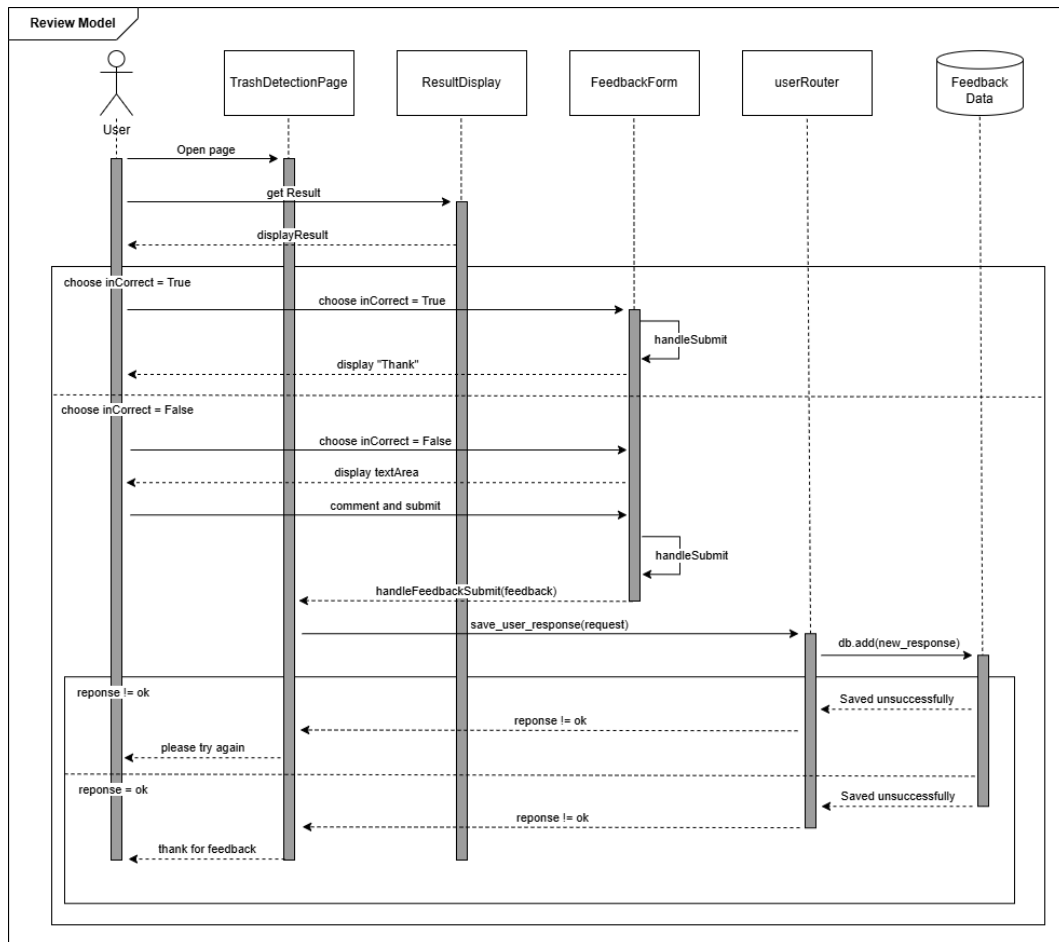
Hình 3.5: Biểu đồ tuần tự người dùng nhận diện rác từ ảnh

Nhận diện rác từ webcam



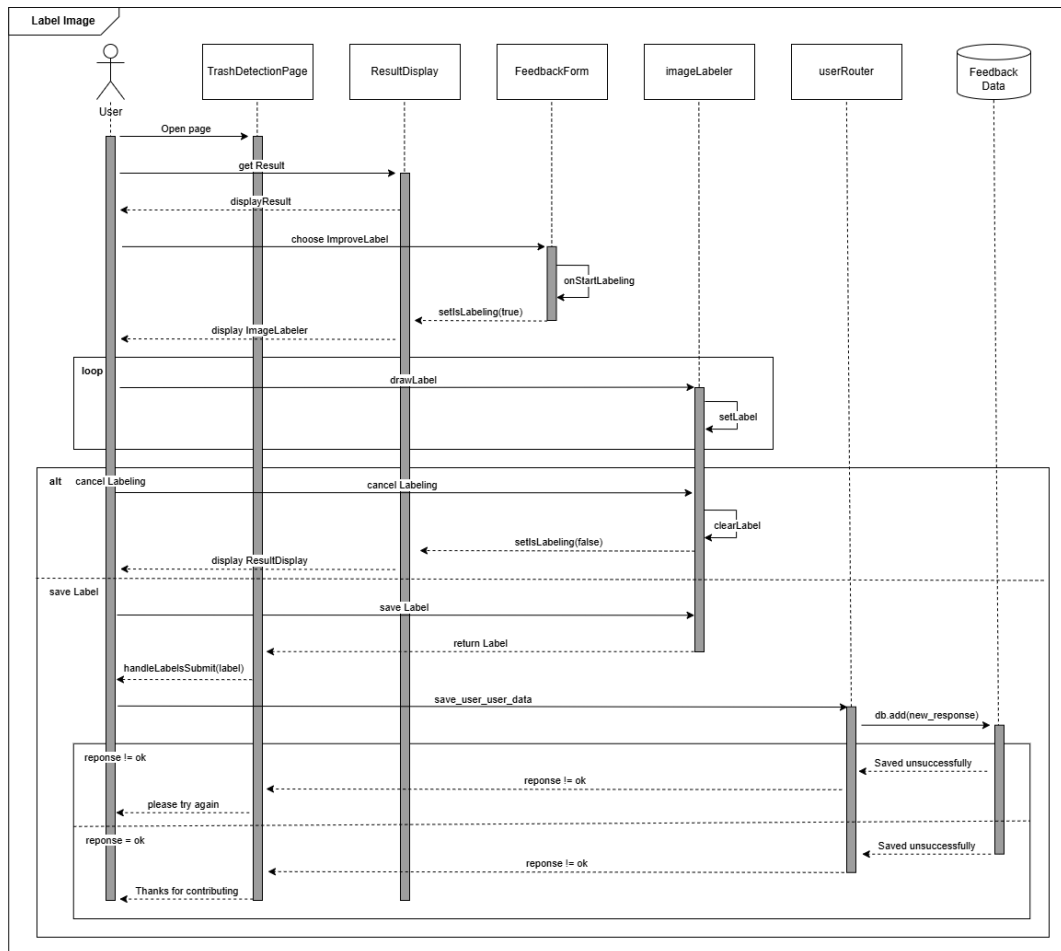
Hình 3.6: Biểu đồ tuần tự người dùng nhận diện rác từ webcam

Đánh giá kết quả



Hình 3.7: Biểu đồ tuần tự người dùng gửi đánh giá

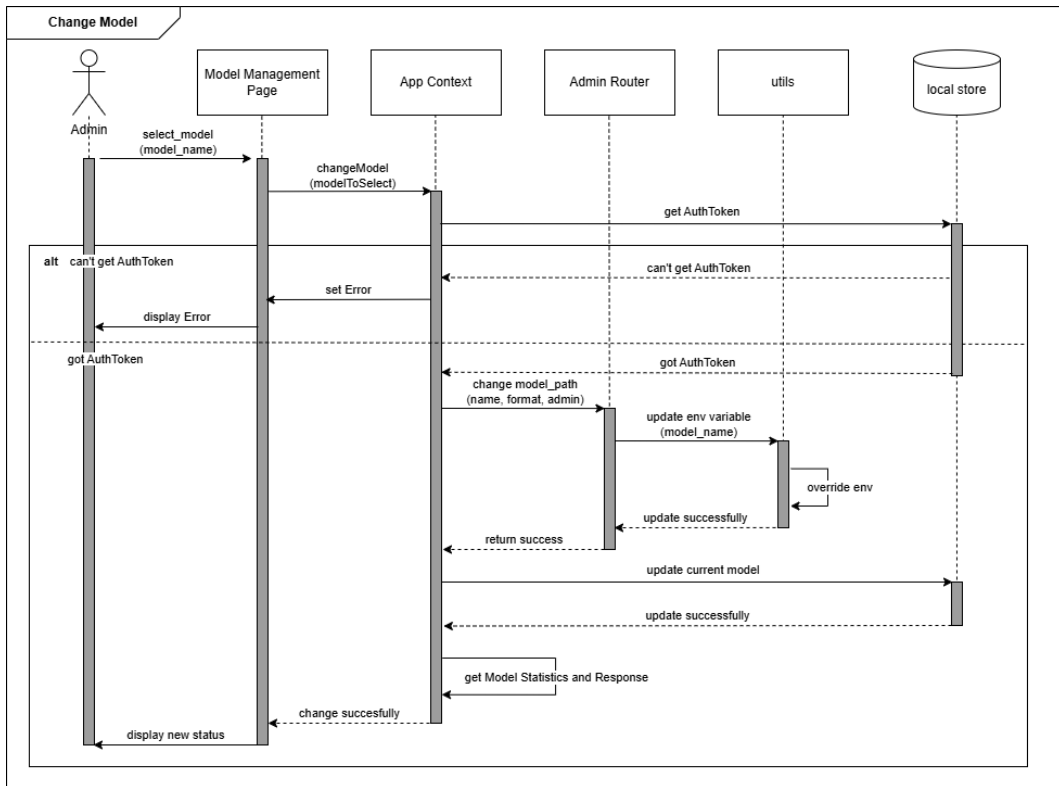
Gán nhãn lại ảnh



Hình 3.8: Biểu đồ tuần tự người dùng gán nhãn lại ảnh

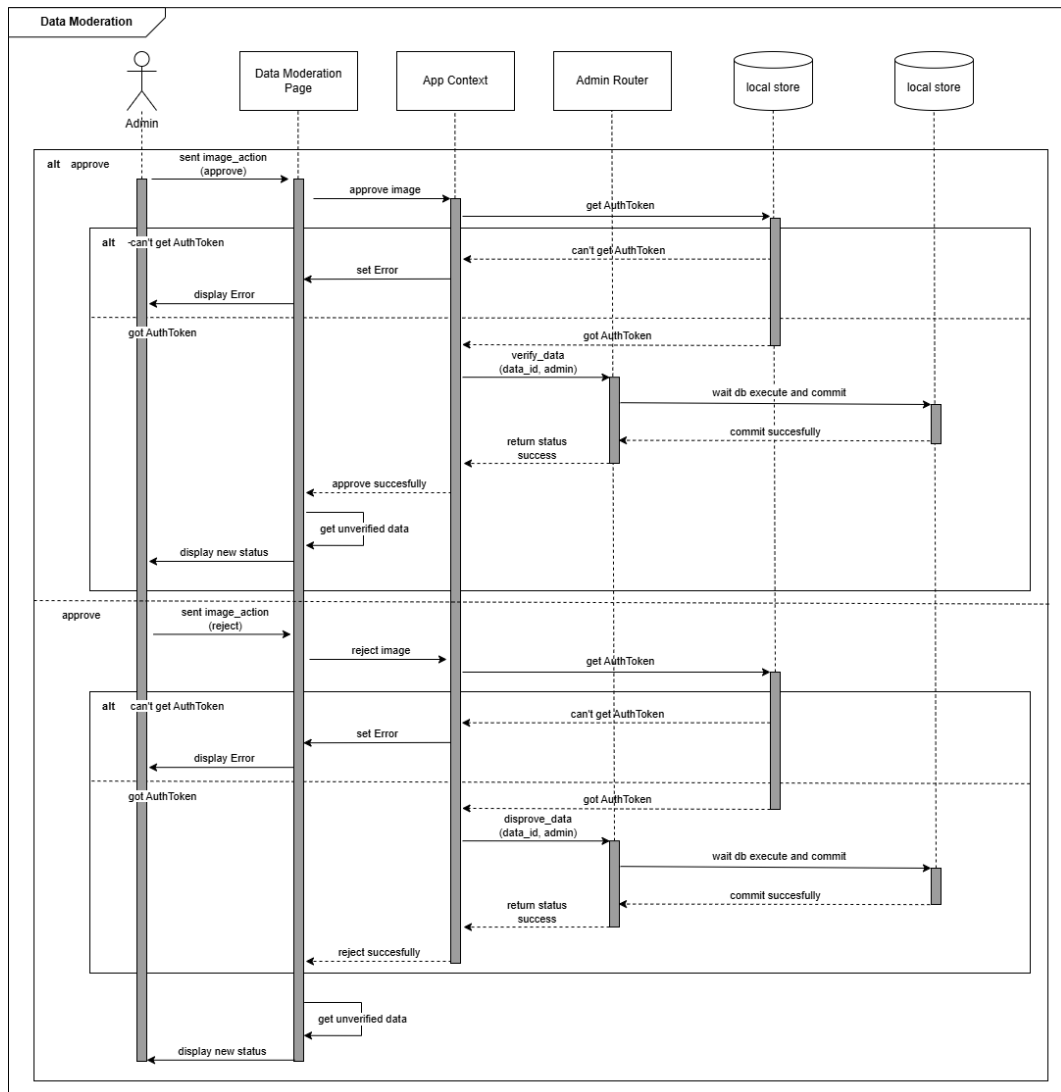
3.4.2 Admin

Thay đổi mô hình



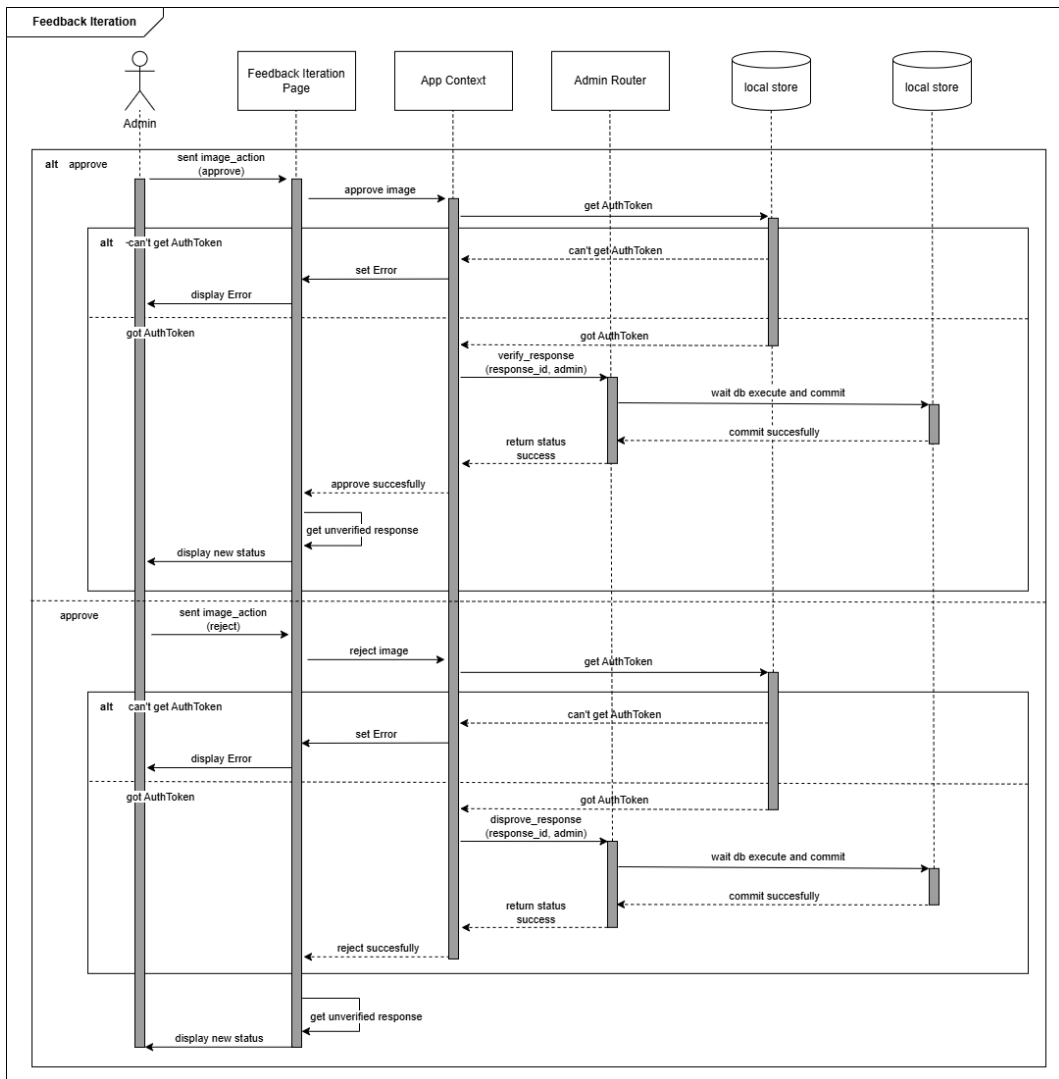
Hình 3.9: Biểu đồ tuần tự Admin thay đổi mô hình

Duyệt các mẫu đóng góp



Hình 3.10: Biểu đồ tuần tự Admin duyệt các mẫu đóng góp

Xét duyệt các bình luận đóng góp



Hình 3.11: Biểu đồ tuần tự Admin xét duyệt các bình luận đóng góp

Chương 4

Kết luận và hướng phát triển

4.1 Tổng kết quá trình thực hiện

Nhóm đã hoàn thiện hệ thống nhận diện rác qua ảnh và webcam sử dụng mô hình Học sâu và triển khai dự án trên Docker để mọi người có thể dễ dàng sử dụng.

4.2 Kết quả đạt được

Kết quả đạt được bao gồm:

- Mô hình có độ chính xác cao.
- Giao diện thân thiện với người dùng và admin.
- Hệ thống hoạt động ổn định, đáp ứng được tất cả các chức năng được đặt ra.
- Triển khai được hệ thống lên Docker, hỗ trợ cho thiết bị có GPU và không có GPU.
- Tăng tốc, cải tiến mô hình giúp giảm đáng kể thời gian xử lý.

4.3 Các hạn chế còn tồn tại

Các hạn chế còn tồn tại bao gồm:

- Khi sử dụng Webcam, mô hình không cho hiệu suất tốt như khi tải ảnh lên.

4.4 Định hướng phát triển tương lai

Định hướng phát triển trong tương lai của hệ thống bao gồm:

- Tăng số loại rác có thể nhận diện.
- Cải thiện khả năng nhận diện khi sử dụng webcam.
- Phát triển ứng dụng mobile.